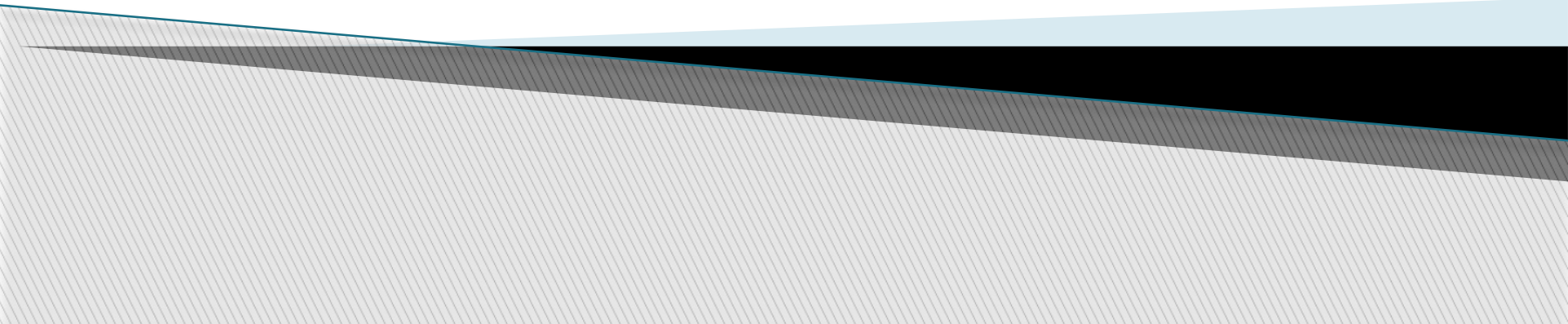


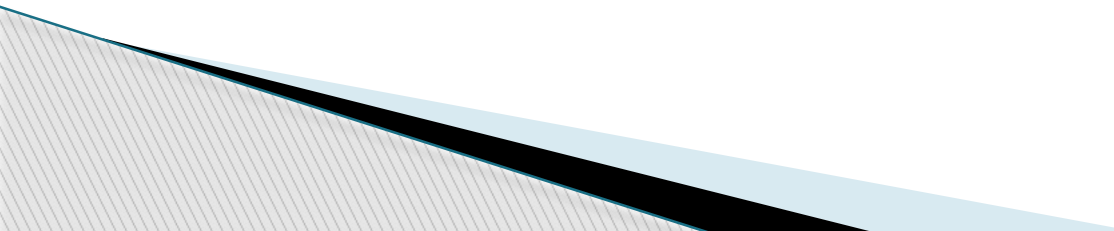
Computer architecture and Organization

Memory Organization



Memory Organization

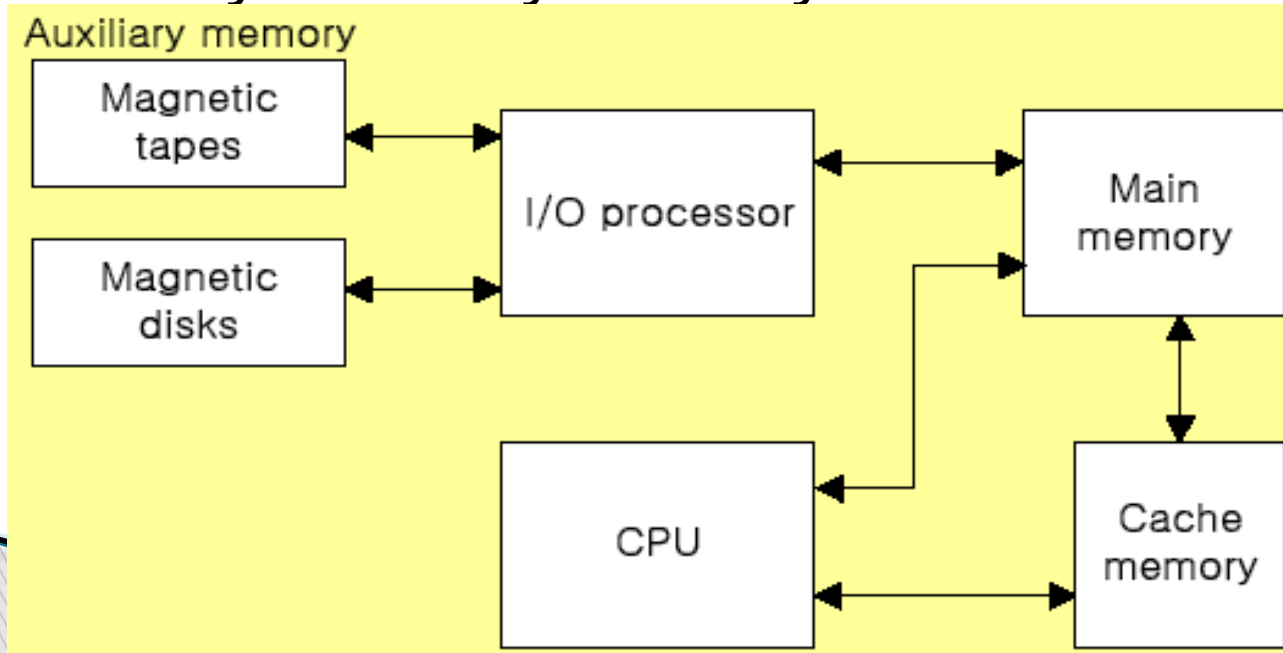
Memory Hierarchy

- Arrangement of different types of memories based on accessibility in computer system
 - Main Memory: memory unit that communicates directly with the CPU (RAM)
 - Auxiliary Memory: device that provide backup storage (Disk Drives)
 - Cache Memory: special very-high-speed memory to increase the processing speed
- 

Memory Organization

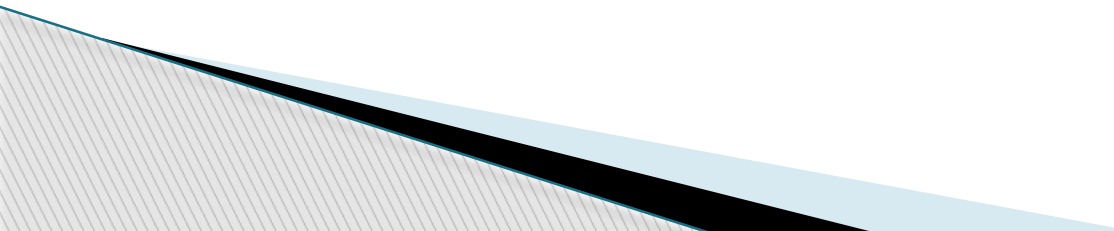
Memory Hierarchy

- ▶ Order according to speed: Cache>Main Memory>Auxiliary memory
- ▶ Order according to size: Cache<Main memory<Auxiliary memory



Memory Organization

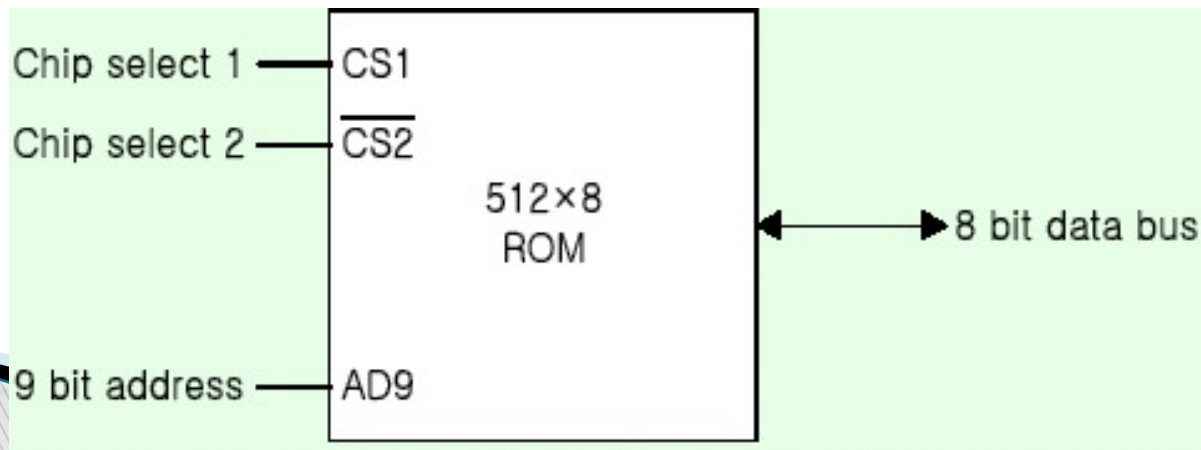
Main Memory

- ▶ Main memory is the only one directly accessible to the CPU.
 - ▶ The CPU continuously reads instructions stored there and executes them as required.
 - ▶ Any data actively operated on is also stored there in uniform manner.
 - ▶ RAM (Random access memory) and ROM (Read only memory) are treated as main memory.
- 

Memory Organization

ROM

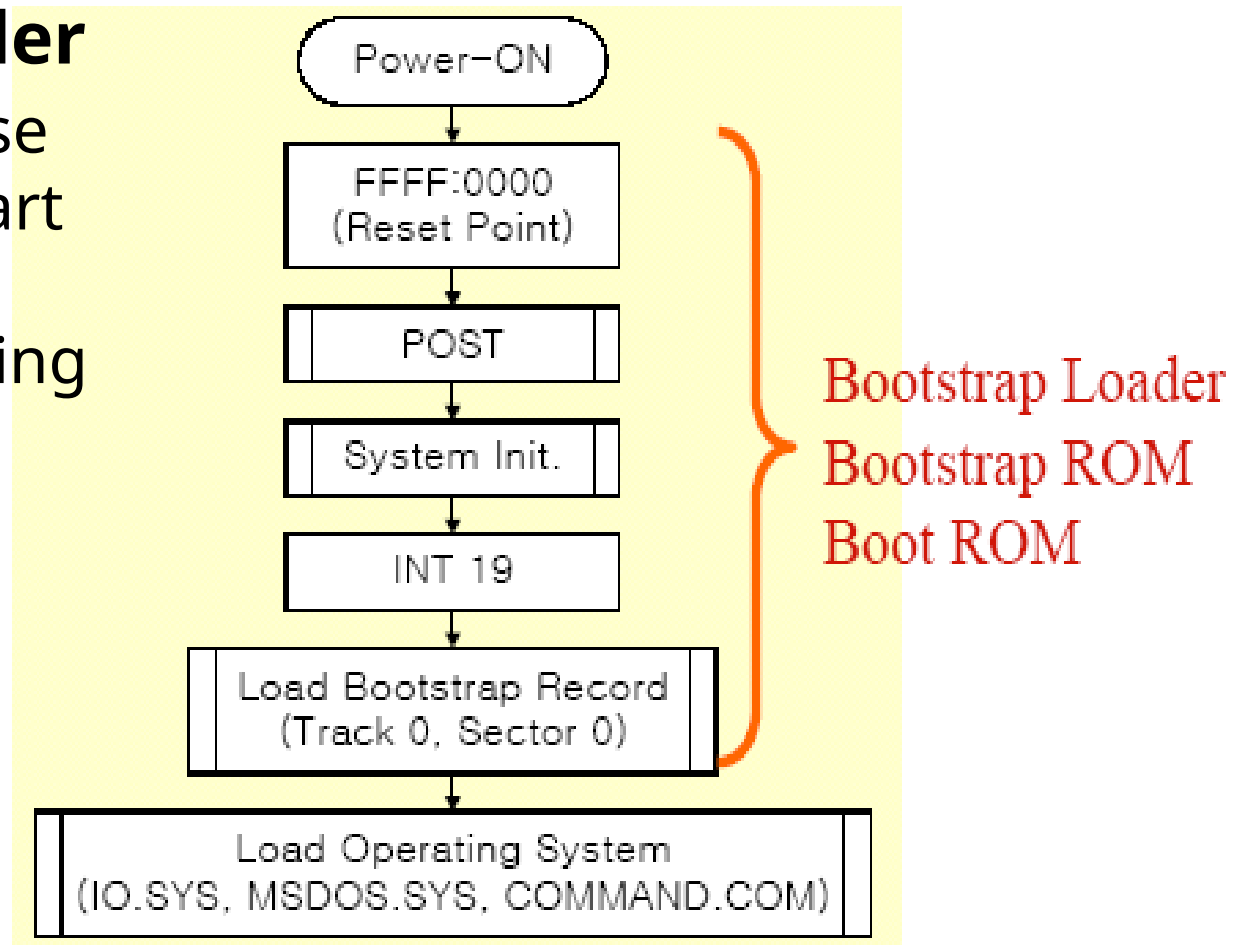
- ▶ Stores a program that helps start up the computer
- ▶ ROM is non-volatile (holds programs and data even after power off)
- ▶ ROM is also direct access but only read
- ▶ The contents of ROM are fixed at the time of manufacture (a typical ROM chip)



Memory Organization

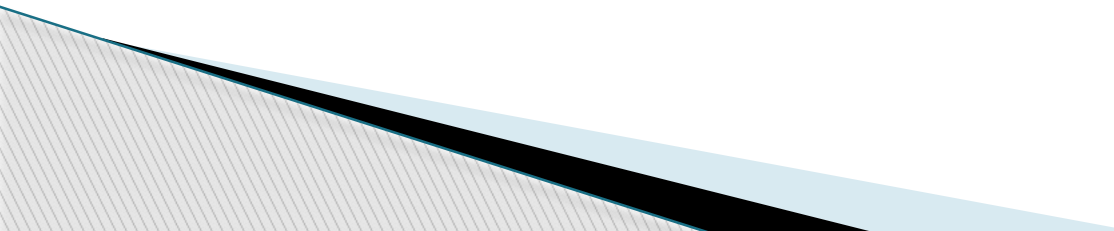
Bootstrap Loader

- ▶ A program whose function is to start the computer software operating when power is turned on



Memory Organization

RAM

- ▶ Its purpose is to temporarily hold programs and data for processing.
 - ▶ Described as being volatile (all data are lost when power off)
 - ▶ It is direct access as it can be both written to or read from in any order
- 

Memory Organization

Types of RAM

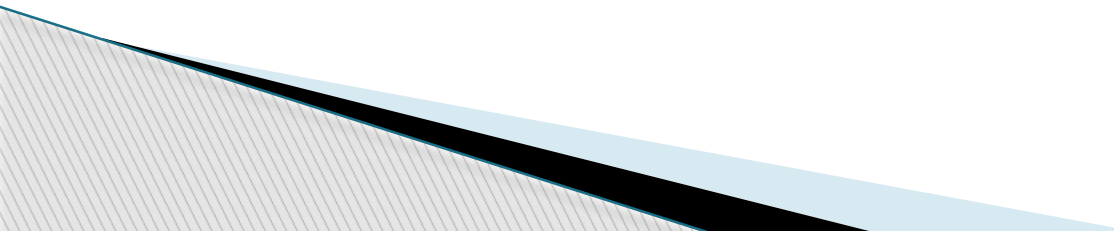
- ▶ Dynamic RAM (DRAM)
- ▶ The RAM which requires the stored information to be periodically re-written, or refreshed, otherwise it would vanish.
- ▶ Static RAM (SRAM)
- ▶ The RAM which never needs to be refreshed as long as power is applied. (It loses its content if power is removed).

Differences between DRAM and SRAM

DRAM	SRAM
1. Store data on capacitive circuit	1. Store data on Flip/Flop
2. Need to refresh after some time	2. No need to refresh
3. Slower than SRAM	3. Faster than DRAM
4. Less expensive	4. More expensive than DRAM

Memory Organization

Memory address map

- ▶ Memory Address Map is a representation of assigned address space for each chip
 - ▶ Assume that a computer system needs 512 bytes of RAM and 512 bytes of ROM
 - ▶ The RAM have 128 byte and need seven address lines, where the ROM have 512 bytes and need 9 address lines
- 

Memory Organization

Memory address map

[illegible]

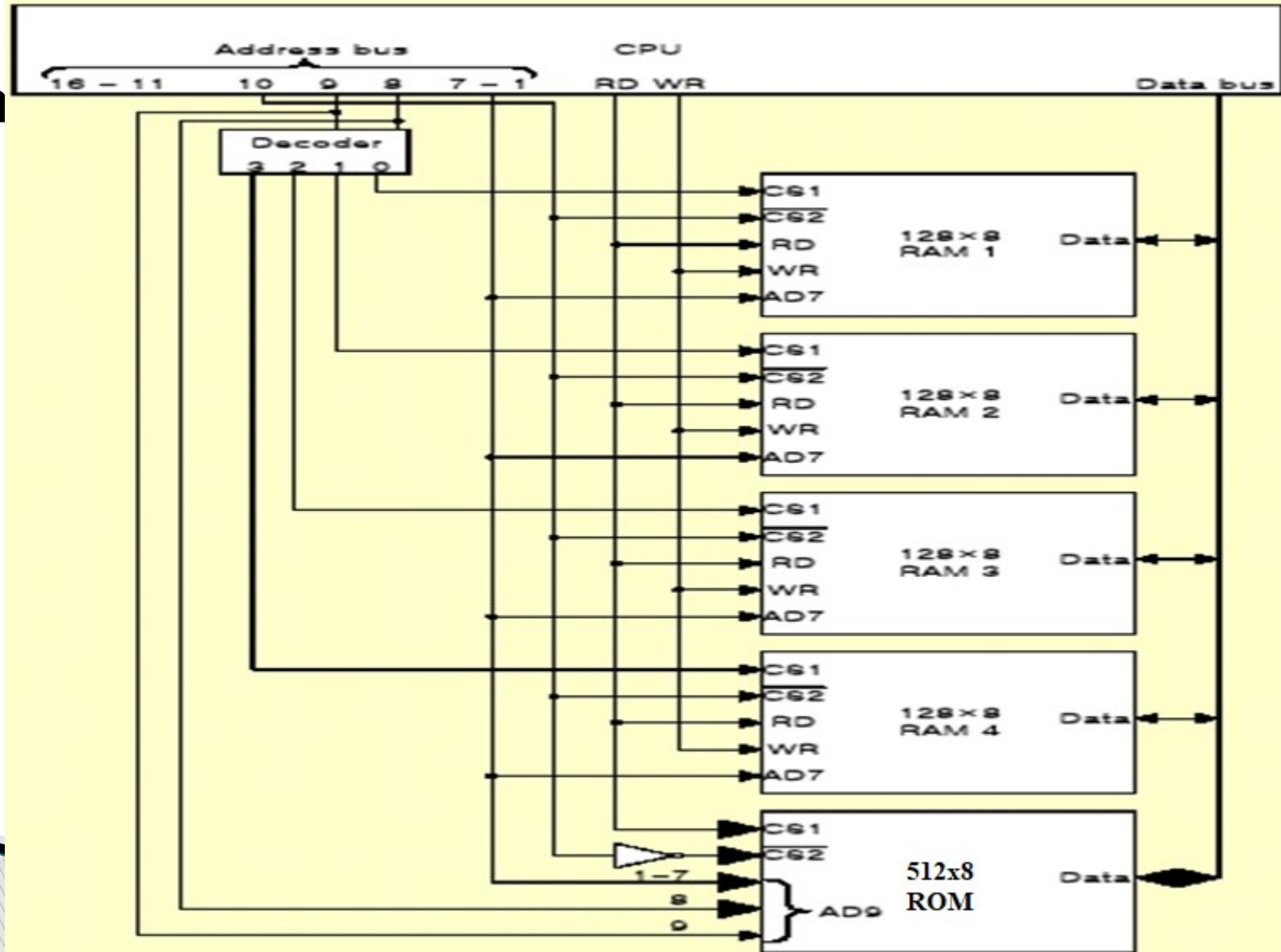
Memory Organization

Memory Connection with CPU

- ▶ The hexadecimal address assigns a range of hexadecimal equivalent address for each chip
- ▶ Line 8 and 9 represent four distinct binary combination to specify which RAM we chose
- ▶ When line 10 is 0, CPU selects a RAM. And when it's 1, it selects the ROM

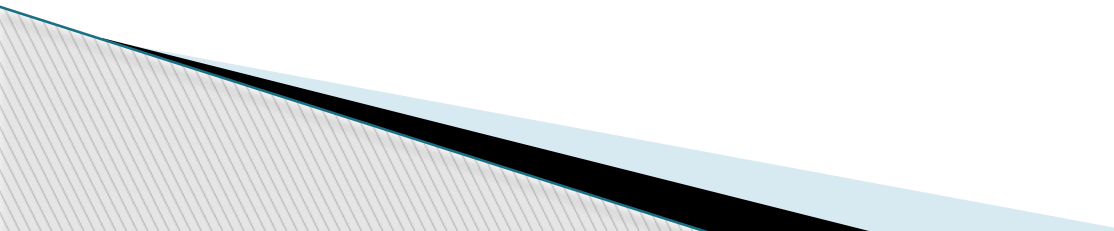
Memory Organization

Men



Memory Organization

Cache memory

- ▶ Small amount of faster memory (SRAM) to speed up computer
 - ▶ Level 1 cache is built within the CPU (internal)
 - ▶ Level 2 cache may be on chip or nearby (external)
 - ▶ Faster for CPU to access than main memory
- 

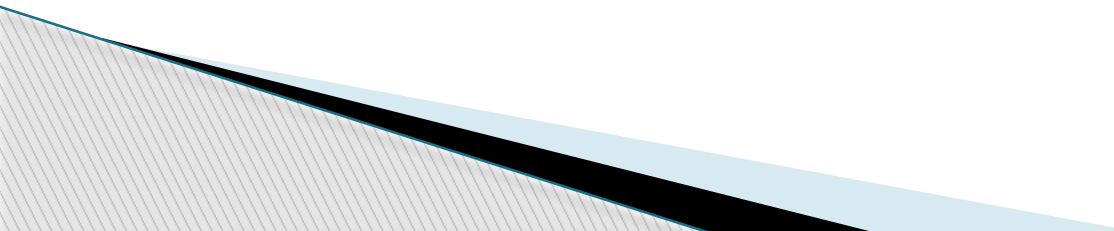
Memory Organization

Cache performance

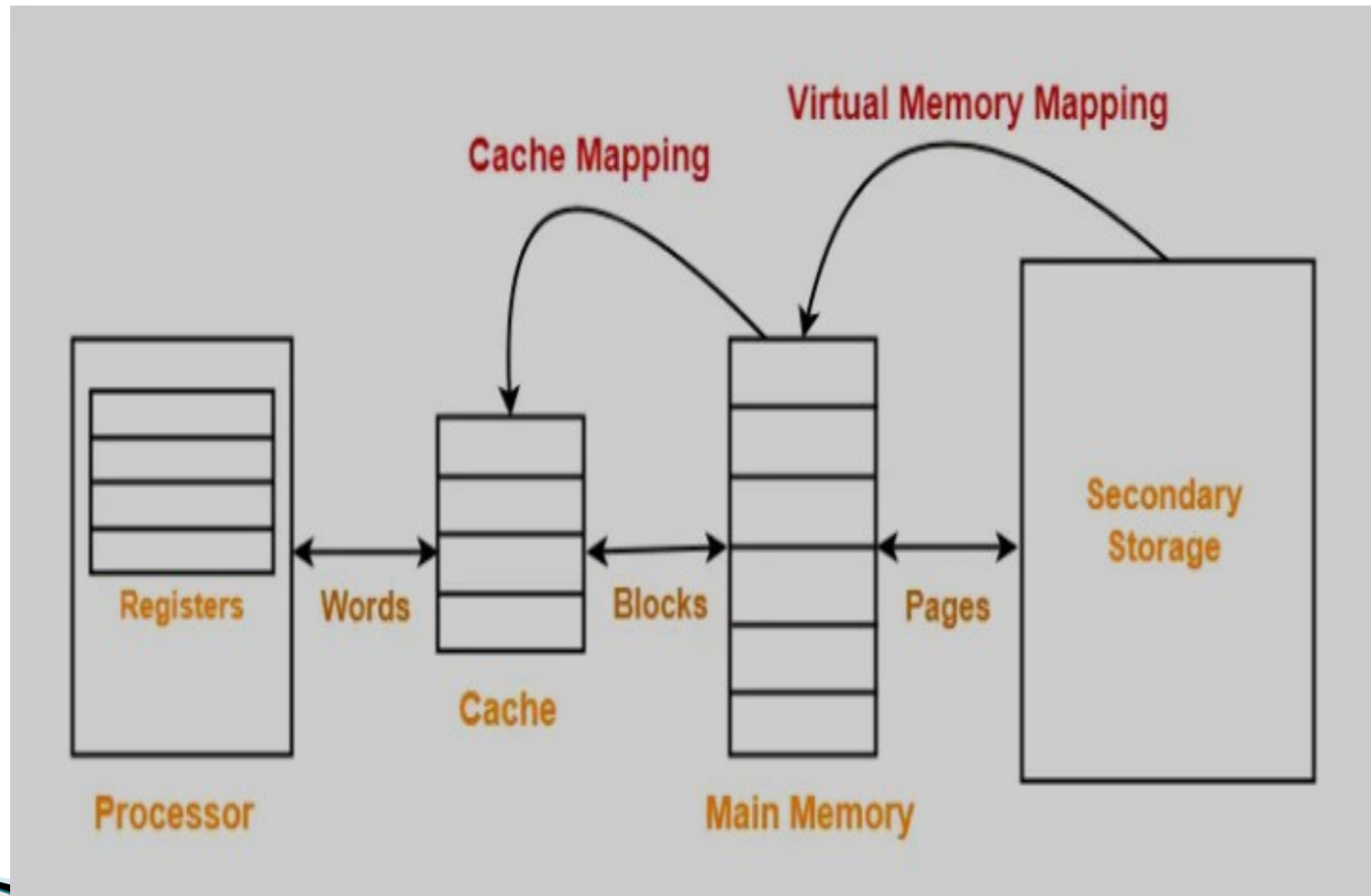
- ▶ Defined as *hit ratio*
- ▶ When the CPU refers to memory and finds the word in cache, it is called *hit*;
- ▶ If the word is not found in cache it counts *miss*.
- ▶ The hit ratio is defined as:
$$\text{hit ratio} = \text{hit} / (\text{hit} + \text{miss}).$$

Memory Organization

Factors of Cache performance

- ▶ Cache size
 - ▶ Mapping method: associative, direct, set-associative
 - ▶ Replace algorithm: LRU, LFU, FIFO
 - ▶ Write policy: write-through, write-back
- 

Memory Organization

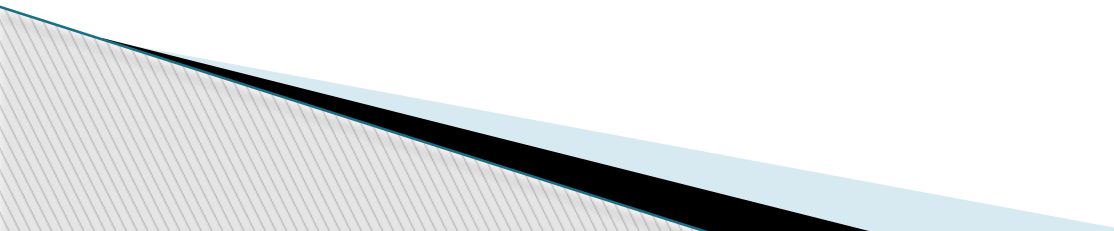


Memory Organization

Cache Replacement Algorithm

- ▶ LRU (Least Recently Used)
- ▶ LFU (Least Frequently Used)
- ▶ FIFO (First-In First-Out)

Writing to Cache

- ▶ Write-through: Data is written to both cache and main memory at the same time.
 - ▶ Write-back: Cache write and set flag to indicate the last writing. Memory updated only when cache block is replaced.
- 

First-In First-Out

Consider the page reference string of size 12: 1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3, with frame size 4 (i.e. maximum 4 pages in a frame).

1	2	3	4	5	1	3	1	6	3	2	3
---	---	---	---	---	---	---	---	---	---	---	---

1	1	1	1	5	5	5	5	5	5	2	2
	2	2	2	2	1	1	1	1	1	1	1
		3	3	3	3	3	3	6	6	6	6
			4	4	4	4	4	4	3	3	3

M	M	M	M	M	M	H	H	M	M	M	H
---	---	---	---	---	---	---	---	---	---	---	---

First-In First-Out

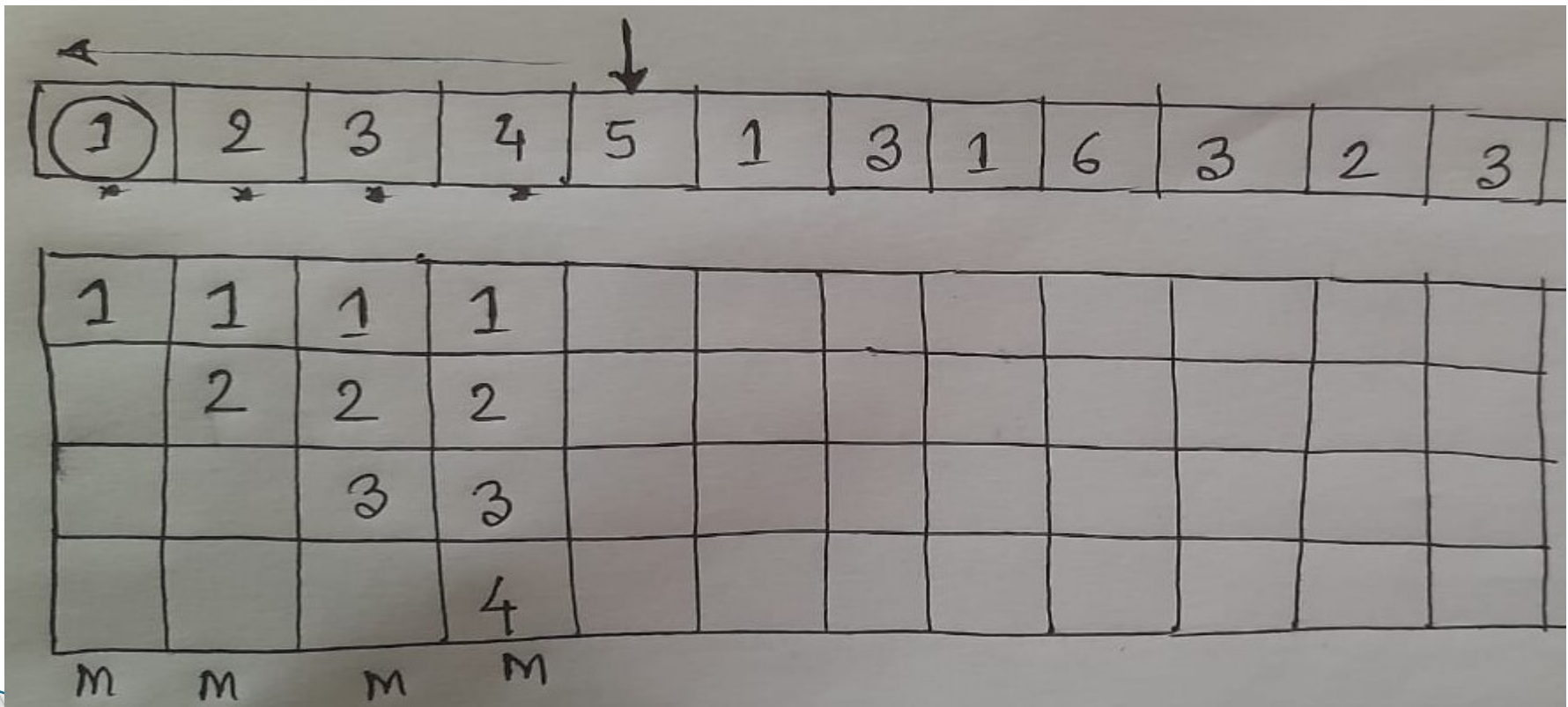
Total Page Fault = 9

Page Fault ratio = total miss/total possible cases

Page Fault ratio = 9/12

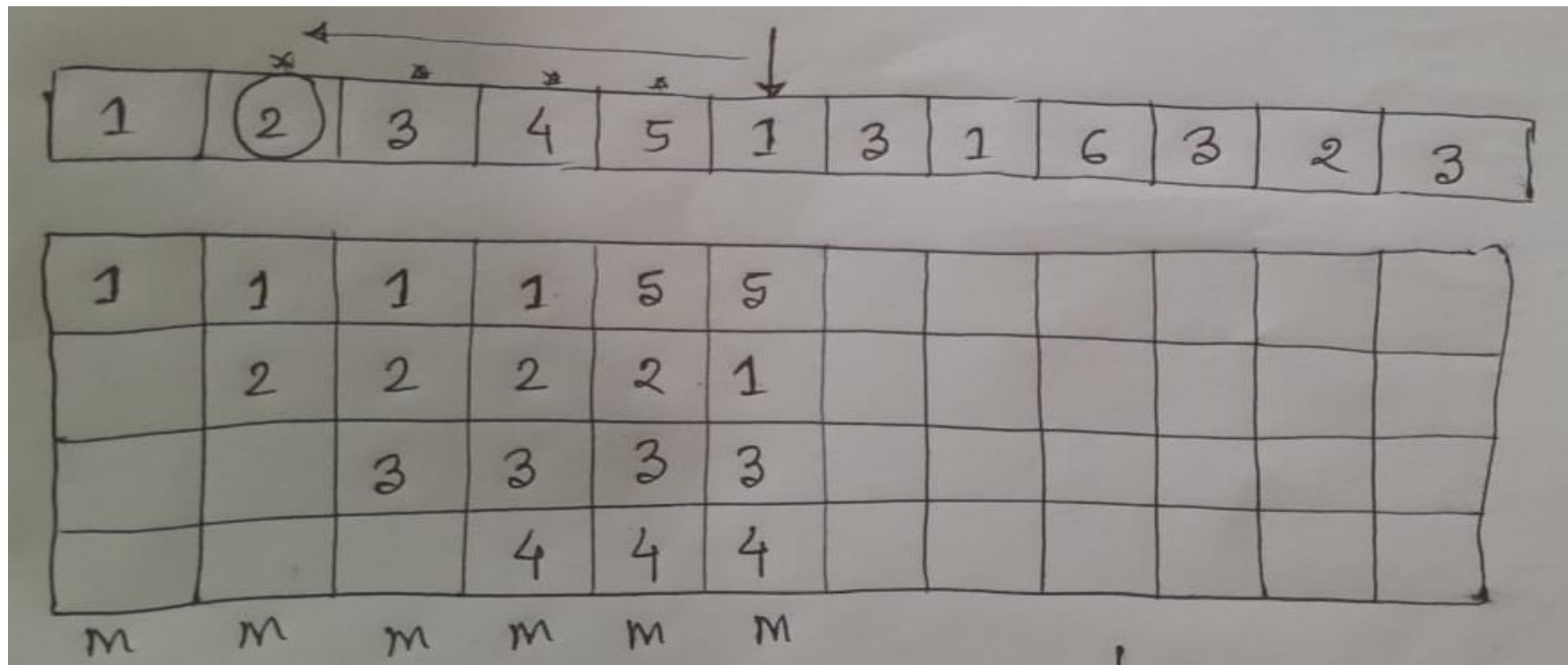
Least Recently Used

Consider the page reference string of size 12: 1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3 with frame size 4 (i.e. maximum 4 pages in a frame).



Least Recently Used

Consider the page reference string of size 12: 1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3 with frame size 4 (i.e. maximum 4 pages in a frame).



Least Recently Used

Consider the page reference string of size 12: 1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3, with frame size 4 (i.e. maximum 4 pages in a frame).

1	2	3	4	5	1	3	1	6	3	2	3
1	1	1	1	5	5	5	5	5			
	2	2	2	2	1	1	1	1			
		3	3	3	3	3	3	3			
			4	4	4	4	4	6			
m	m	m	m	m	m	H	H				

Least Recently Used

1	2	3	4	5	1	3	1	6	3	2	3
---	---	---	---	---	---	---	---	---	---	---	---

1	1	1	1	5	5	5	5	5	5	2	2
	2	2	2	2	1	1	1	1	1	1	1
		3	3	3	3	3	3	3	3	3	3
			4	4	4	4	4	6	6	6	6

M	M	M	M	M	M	H	H	M	H	M	H
---	---	---	---	---	---	---	---	---	---	---	---

Least Recently Used

Total Page Fault = 8

Page Fault ratio = total miss/total possible cases

Page Fault ratio = 8/12

Least Frequently Used

Consider the page reference string of size 10:
3,5,6,6,1,1,4,2,3,4 with frame size 3(i.e. maximum 3
pages in a frame).

3	5	6	6	1	1	4	2	3	4
3	3	3	3	1					
	5	5	5	5					
		6	6	6					
M	M	M	H	M					

	3	5	6	1	4	2
Freq.	1	1	2	0 / 1	0	0

Least Frequently Used

Consider the page reference string of size 10:
3,5,6,6,1,1,4,2,3,4 with frame size 3(i.e. maximum 3
pages in a frame).

3	5	6	6	1	1	4	2	3	4
3	3	3	3	1	1	1			
	5	5	5	5	5	4			
		6	6	6	6	6			
M	M	M	H	M	H	M			

	3	5	6	1	4	2
Freq.	0	1	2	1/2	0	0

Least Frequently Used

Consider the page reference string of size 10:
3,5,6,6,1,1,4,2,3,4 with frame size 3(i.e. maximum 3 pages in a frame).

3	5	6	6	1	1	4	2	3	4
3	3	3	3	1	1	1	1		
	5	5	5	5	5	4	2		
		6	6	6	6	6	6		
M	M	M	H	M	H	M	M		

	3	5	6	1	4	2
Freq.	0	0	2	2	0/1->0	0->1

Least Frequently Used

Consider the page reference string of size 10:
3,5,6,6,1,1,4,2,3,4 with frame size 3(i.e. maximum 3
pages in a frame).

3	5	6	6	1	1	4	2	3	4
3	3	3	3	1	1	1	1	1	
	5	5	5	5	5	4	2	3	
		6	6	6	6	6	6	6	
M	M	M	H	M	H	M	M	M	

	3	5	6	1	4	2
Freq.	0+1	0	2	2	0	1->0

Least Frequently Used

Consider the page reference string of size 10:
3,5,6,6,1,1,4,2,3,4 with frame size 3(i.e. maximum 3
pages in a frame).

3	5	6	6	1	1	4	2	3	4
3	3	3	3	1	1	1	1	1	1
	5	5	5	5	5	4	2	3	4
		6	6	6	6	6	6	6	6
M	M	M	H	M	H	M	M	M	M

	3	5	6	1	4	2
Freq.	1->0	0	2	2	0+1	0

Least Frequently Used

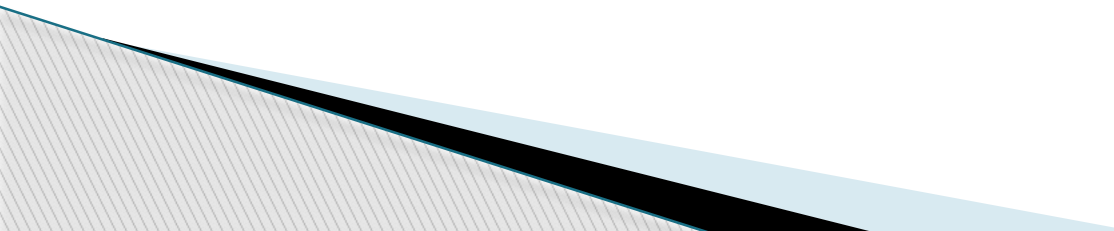
Total Page Fault = 8

Page Fault ratio = total miss/total possible cases

Page Fault ratio = 8/10

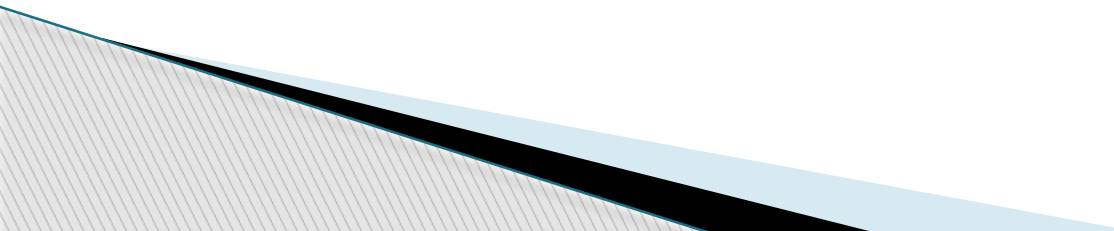
Memory Organization

Secondary Storage

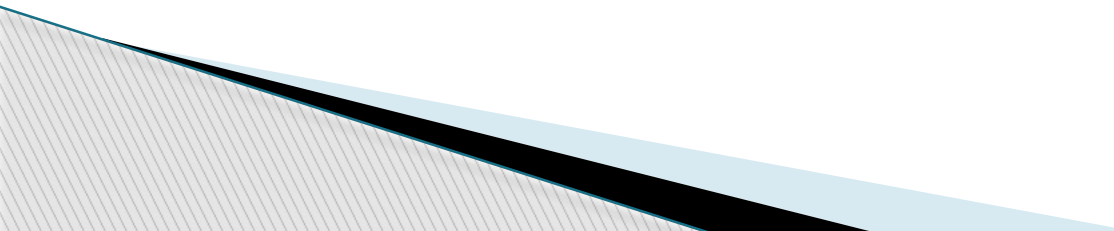
- ▶ Secondary storage (auxiliary storage) is not directly accessible by the CPU.
 - ▶ The computer usually uses its I/O channels to access secondary storage
 - ▶ Transfers the desired data using intermediate area in primary storage.
 - ▶ Secondary storage does not lose data when the device is powered off - non-volatile.
- 

Memory Organization

Virtual Memory

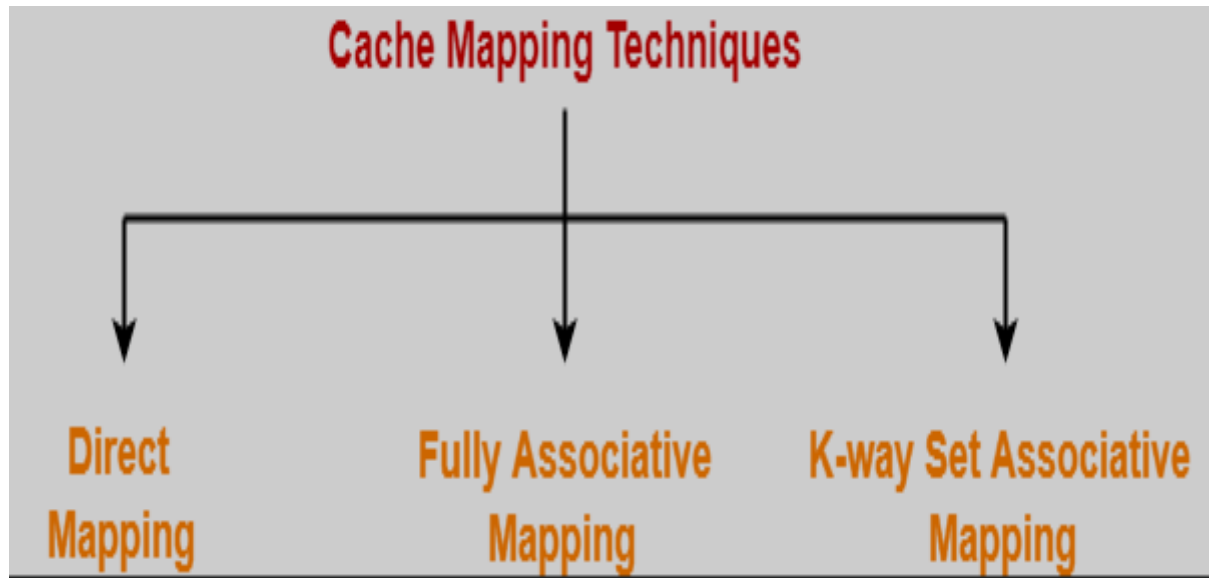
- ▶ Uses backing storage e.g. hard disk as a temporary location for programs and data where insufficient RAM available
 - ▶ Swaps programs and data between the hard-disk and RAM as the CPU requires them for processing
 - ▶ A cheap method of running large or many programs on a computer system
 - ▶ Cost is speed: the CPU can access RAM in nanoseconds but hard-disk in milliseconds
 - ▶ Virtual memory is much slower than RAM
- 

Memory Organization

- ▶ **Memory Management**
 - ▶ Collection of hardware and software for managing the various programs residing in memory.
 - ▶ The basic components of a memory management unit are:
 - A facility for dynamic storage relocation that maps logical memory references into physical memory addresses
 - A provision for sharing common programs stored in memory by different users
 - Protection of information against unauthorized access between users and preventing users from changing operating systems functions.
- 

Memory Organization

- ▶ **Cache mapping techniques**



Memory Organization

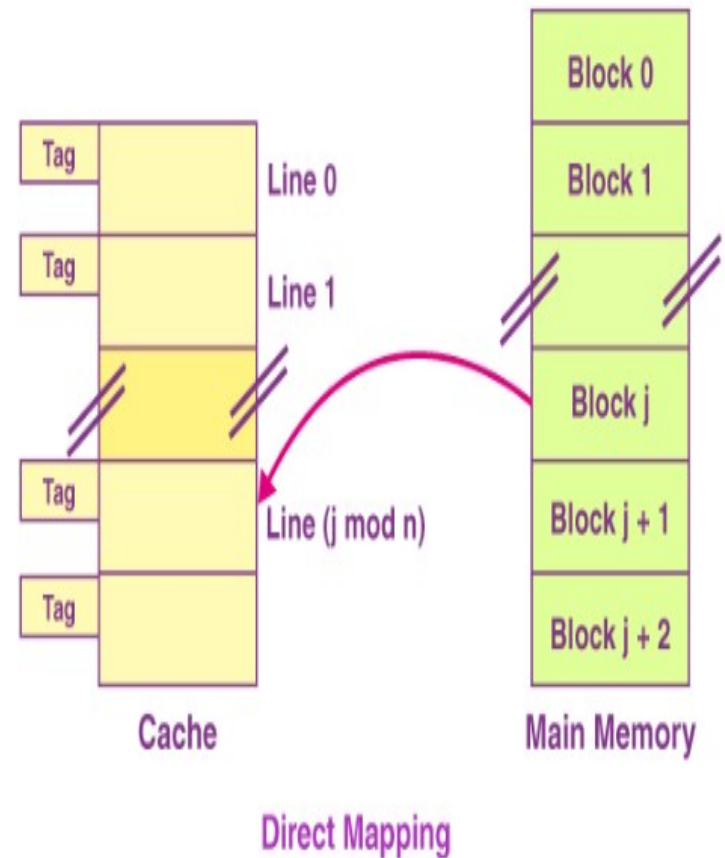
- ▶ Direct Mapping
 - ❑ A particular block of main memory can map only to a particular line of the cache.
 - ❑ The line number of cache to which a particular block can map is given by-
 - ❑ **Cache line number= (Main Memory Block Address) Modulo (Number of lines in Cache)**

Memory Organization

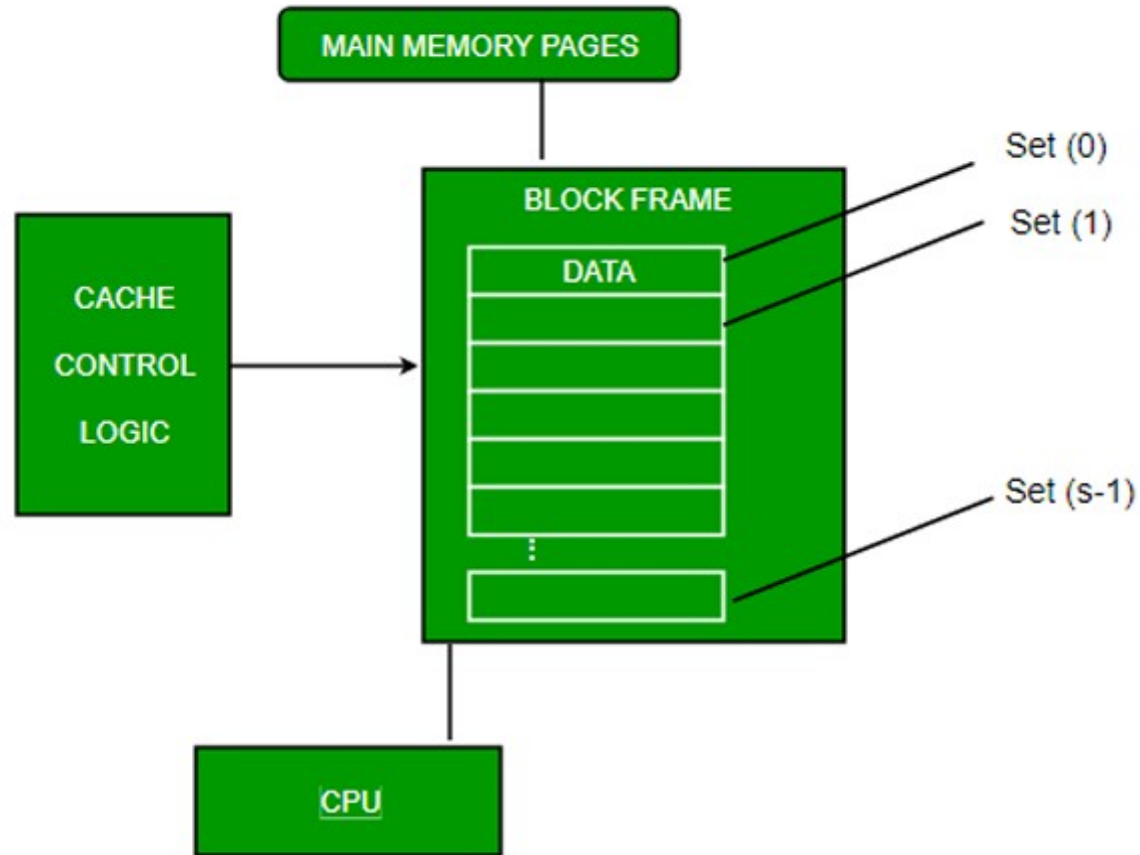
- ▶ Direct Mapping

Consider cache memory is divided into 'n' number of lines.

Then, block 'j' of main memory can map to line number $(j \bmod n)$ only of the cache.



Memory Organization

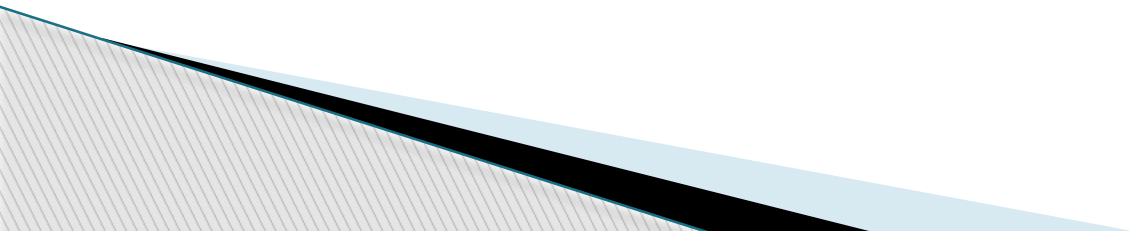


Memory Organization

- ▶ In direct mapping,
There is no need of any replacement algorithm.

This is because a main memory block can map only to a particular line of the cache.

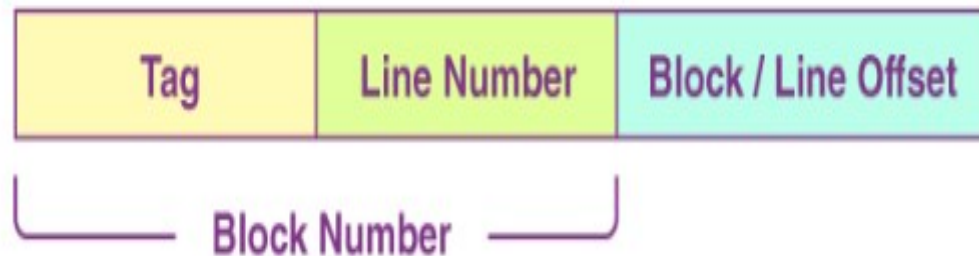
Thus, the new incoming block will always replace the existing block (if any) in that particular line.



Memory Organization

□ Division of Physical Address

physical address is the actual hardware-level location in the computer's main memory (RAM).



Memory Organization

□ Division of Physical Address

- MM Size: 4 GB
 - Cache Size: 1 MB
 - Block Size: 4 KB
-
- ▶ P.A. bits split?
 - ▶ Tag directory size?

Memory Organization

□ Division of Physical Address

▶ MM Size = 4 GB

$$= 2^2 \times 2^{30} \text{B}$$

$$= 2^{(2+30)} \text{B}$$

$$= 2^{32} \text{B}$$

$$\therefore \text{No. of P.A. bits} = \log_2 2^{32} = 32$$

Memory Organization

□ Division of Physical Address

- ▶ Block Size = 4 KB
= $2^2 \times 2^{10} \text{B}$
= 2^{12}B [Block offset=12 bits]

No. of Blocks in MM = $2^{32}/2^{12}=2^{20}$

∴ Block number bits = $\log_2 2^{20}=20$

Memory Organization

- ▶ Division of Physical Address

- ▶ Cache Size = 1 MB
 $= 1 \times 2^{20} \text{B}$
 $= 2^{20} \text{B}$

No. of Lines in Cache = $2^{20}/2^{12} = 2^8$

$\therefore$ Line number bits = $\log_2 2^8 = 8$

No. of Tag bits: P.A. bits – (Line no. bits + offset)
 $= 32 - (8 + 12)$
= 12 bits

Memory Organization

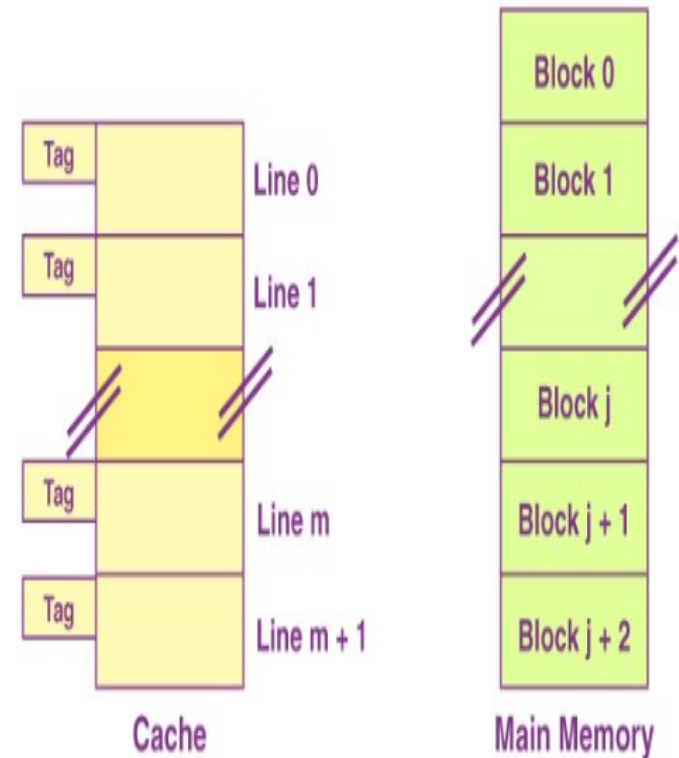
▶ Fully Associative Mapping

In fully associative mapping,

- ❑ A block of main memory can map to **any line of the cache** that is freely available at that moment.
- ❑ This makes fully associative mapping more flexible than direct mapping.

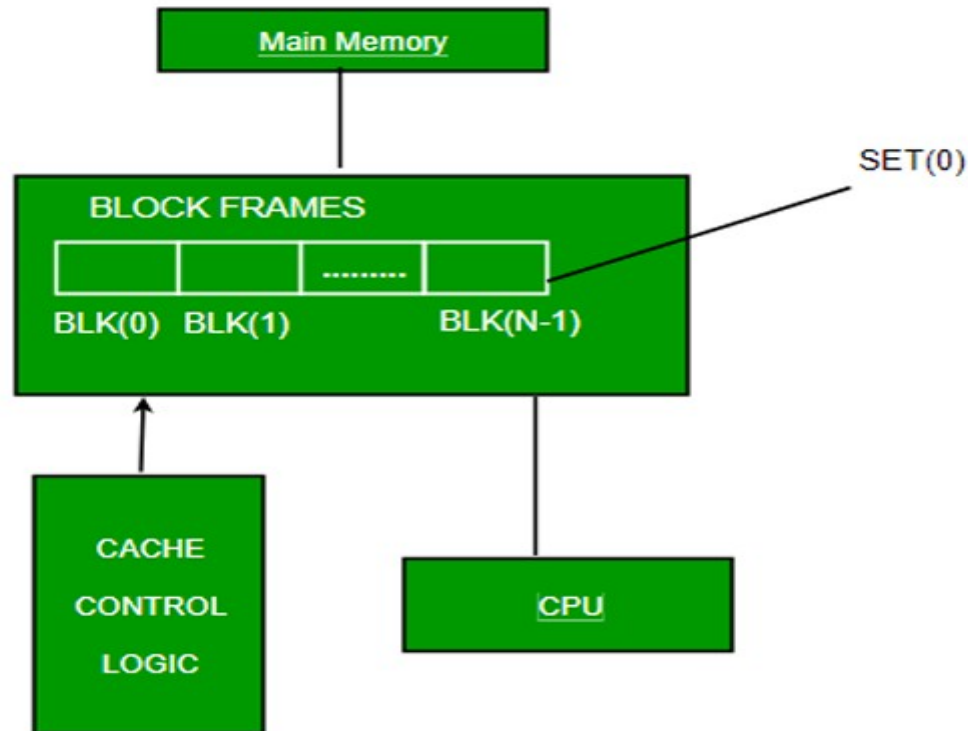
Memory Organization

- ▶ **Fully Associative Mapping**
- ▶ All the lines of cache are freely available.
- ▶ Thus, any block of main memory can map to any line of the cache.
- ▶ Had all the cache lines been occupied, then one of the existing blocks will have to be replaced



Memory Organization

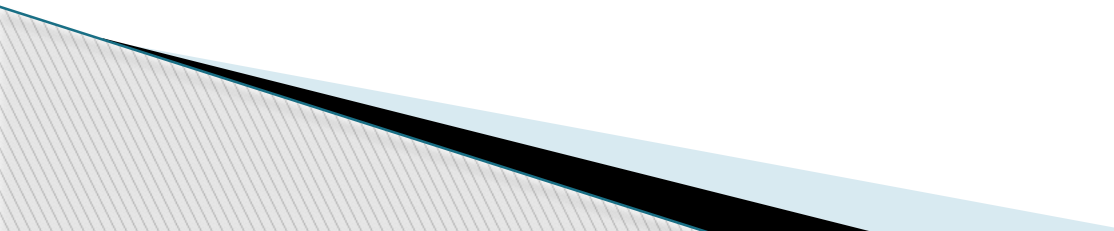
- ▶ Fully Associative Mapping



Memory Organization

► Need of Replacement Algorithm

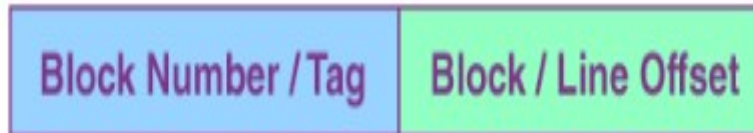
In fully associative mapping,

- ❑ A replacement algorithm is required.
 - ❑ Replacement algorithm suggests the block to be replaced if all the cache lines are occupied.
 - ❑ Thus, replacement algorithm like FCFS Algorithm, LRU Algorithm etc is employed.
- 

Memory Organization

► Division of Physical Address-

In fully associative mapping, the physical address is divided as-



Memory Organization

- ▶ Example 01: Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find-
 - a. Number of bits in tag
 - b. Tag directory size

Memory Organization

- ▶ Example:02 Consider a fully associative mapped cache of size 512 KB with block size 1 KB. There are 17 bits in the tag. Find-
 - a. Size of main memory
 - b. Tag directory size

Memory Organization

► K-way Set Associative Mapping

In k-way set associative mapping,

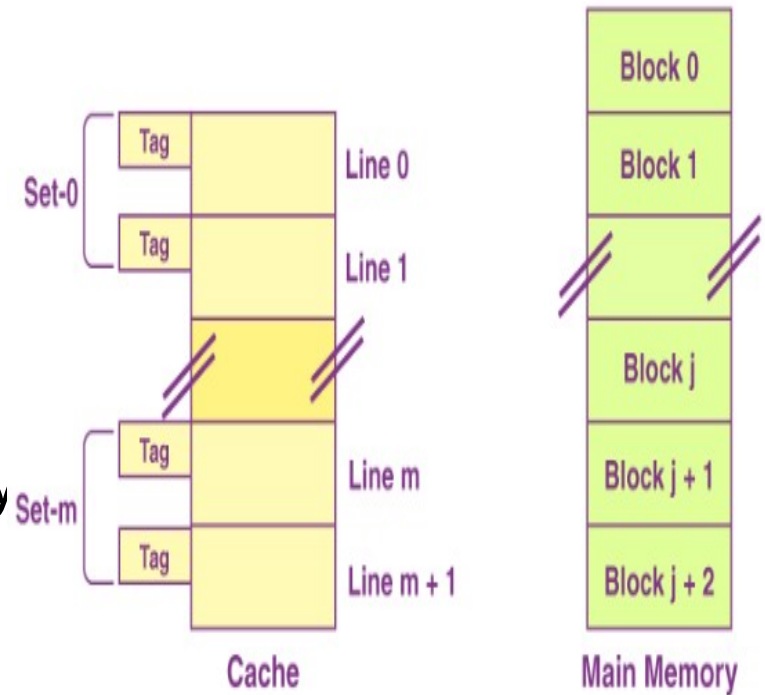
- ❑ Cache lines are grouped into sets where each set contains k number of lines.
- ❑ A particular block of main memory can map to only one particular set of the cache.
- ❑ However, within that set, the memory block can map any cache line that is freely available.
- ❑ The set of the cache to which a particular block of the main memory can map is given by-

Cache set number= (Main Memory Block Address) Modulo (Number of sets in Cache)

Memory Organization

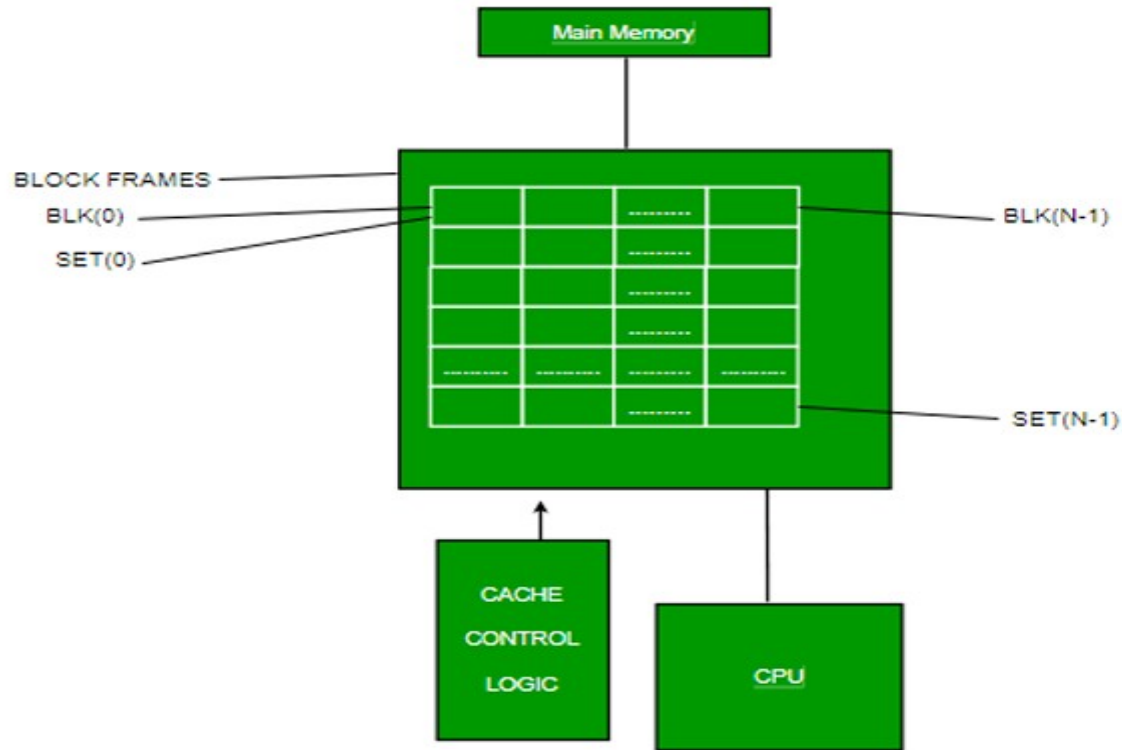
▶ K-way Set Associative Mapping

- ▶ $k = 2$ suggests that each set contains two cache lines.
- ▶ Since cache contains 6 lines, so number of sets in the cache = $6 / 2 = 3$ sets.
- ▶ Block 'j' of main memory can map to set number $(j \bmod 3)$ only of the cache.
- ▶ Within that set, block 'j' can map to any cache line that is freely available at that moment.
- ▶ If all the cache lines are occupied, then one of the existing blocks will have to be replaced.

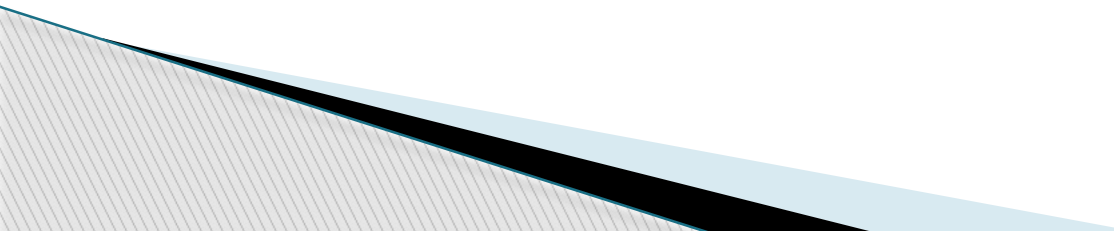


Memory Organization

► K-way Set Associative Mapping



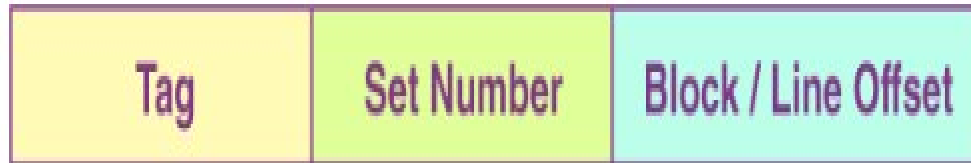
Memory Organization

- ▶ **Need of Replacement Algorithm-**
 - ▶ Set associative mapping is a combination of direct mapping and fully associative mapping.
 - ▶ It uses fully associative mapping within each set.
 - ▶ Thus, set associative mapping requires a replacement algorithm.
- 

Memory Organization

► Division of Physical Address

- In set associative mapping, the physical address is divided as-



Special Case: If $k = 1$, then k -way set associative mapping becomes direct mapping

Memory Organization

► Logical Address:

A logical address is generated by the CPU while a program runs. It represents the address from the process's perspective and does not exist physically, hence it is also called a virtual address.

- The logical address space is the set of all logical addresses a process can generate.
- Programs use logical addresses to reference memory, and the MMU translates them into physical addresses when accessing actual memory.

Memory Organization

► Physical Address:

A physical address is the real location in main memory (RAM) where data or instructions are stored. The physical address space consists of all physical addresses corresponding to logical addresses.

- The MMU performs address translation using a page table, mapping each logical page to a physical frame.
- This allows processes to access memory transparently, without knowing actual memory locations.

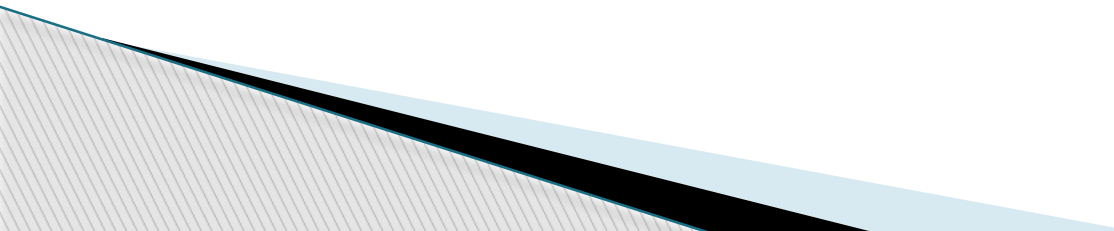
Memory Organization

▶ **Paging:**

Paging is a memory management technique in which a process is divided into fixed-size blocks called pages, and physical memory is divided into frames of the same size. Pages are loaded from secondary storage into main memory as needed.

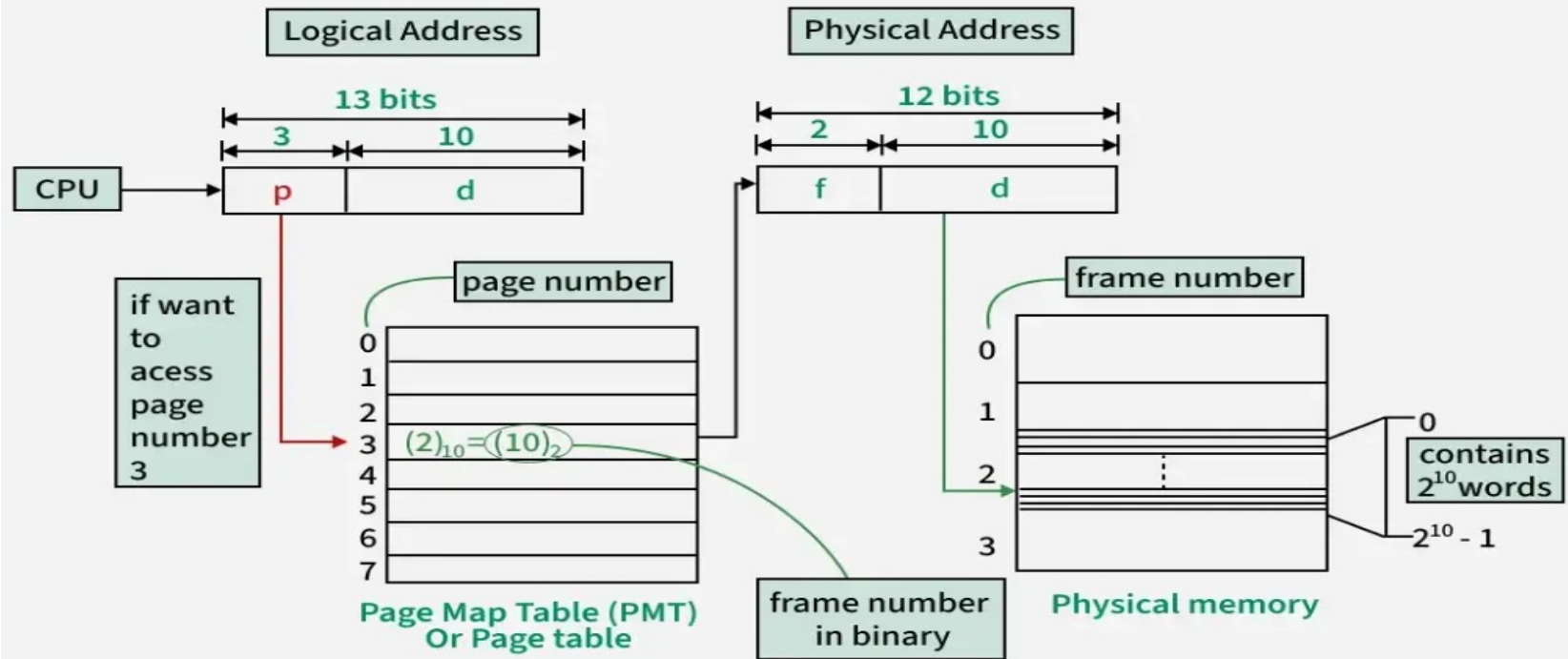
- A process does not need to be stored in contiguous memory locations.
- The entire process does not need to be loaded into main memory at once; only required pages are loaded on demand. This allows efficient utilization of memory and supports execution of large programs.
- Memory allocation is simplified because memory is managed in fixed-size pages and frames.

Memory Organization

- ▶ **Paging:**
 - ▶ The Physical Address Space is conceptually divided into a number of fixed-size blocks, called frames.
 - ▶ The Logical Address Space is also split into fixed-size blocks, called pages.
 - ▶ Page Size = Frame Size
- 

Memory Organization

► Paging:



Memory Organization

► Steps of Paging

- Paging is a memory management technique that divides logical memory into fixed-size **pages** and physical memory into fixed-size **frames**. The process of address translation occurs as follows:
- **CPU Generates a Logical Address**
 - When a process executes, the CPU produces a logical (virtual) address.
 - The logical address consists of:
 - **Page Number (p)**
 - **Page Offset (d)**
- **Page Number Lookup**
 - The page number is used as an index into the process's **page table**.
 - The page table stores the corresponding frame number for each page.

Memory Organization

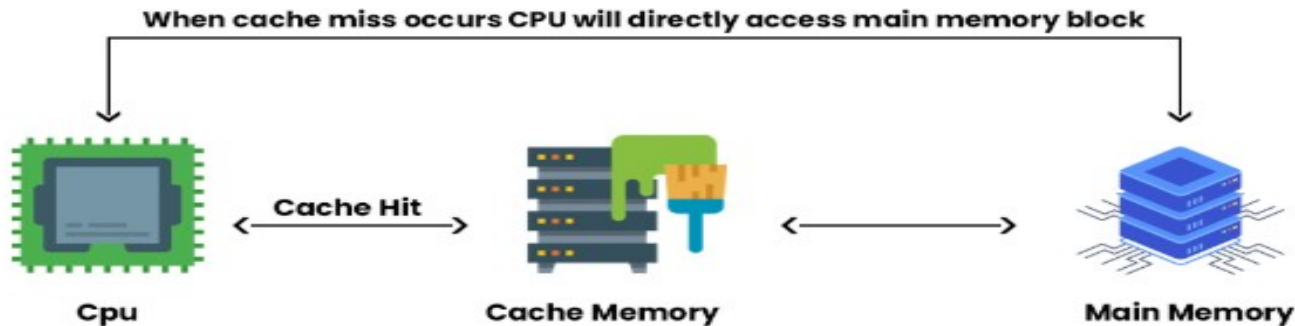
- ▶ **Steps of Paging**
- ▶ **Frame Number Retrieval**
 - The operating system retrieves the frame number (**f**) associated with the page number (**p**) from the page table.
- ▶ **Physical Address Formation**
 - The frame number is combined with the page offset.
 - Physical Address = Frame Number + Offset
- ▶ **Memory Access**
 - The generated physical address is used to access the required location in main memory.
- ▶ **Data Transfer**
 - The requested instruction or data is fetched from memory and provided to the CPU.
- ▶ **Address Translation Formula**
Physical Address=(Frame Number×Frame Size)+Offset

Memory Organization

- ▶ **Steps of Paging**
- ▶ **Frame Number Retrieval**
 - The operating system retrieves the frame number (**f**) associated with the page number (**p**) from the page table.
- ▶ **Physical Address Formation**
 - The frame number is combined with the page offset.
 - Physical Address = Frame Number + Offset
- ▶ **Memory Access**
 - The generated physical address is used to access the required location in main memory.
- ▶ **Data Transfer**
 - The requested instruction or data is fetched from memory and provided to the CPU.
- ▶ **Address Translation Formula**
Physical Address=(Frame Number×Frame Size)+Offset

Memory Organization

- ▶ Effective memory access time:



- ▶ Average memory access time = Hit ratio * Cache memory access time + (1 - Hit ratio) * (Cache Memory access time + Main memory access time)

Memory Organization

- ▶ **Effective memory access time:**

- ▶ **Example:** In two level hierarchy, the top level has an access time of 10ns and bottom level has an access time of 50ns, the hit rate on the top level is 90%. what is the average memory access time required?

- ▶ **Solution:**

Given, Cache memory access time (top level)= 10ns

Main memory access time (bottom level) = 50ns

Hit ratio=0.9, miss ratio=(1-0.9)=0.1

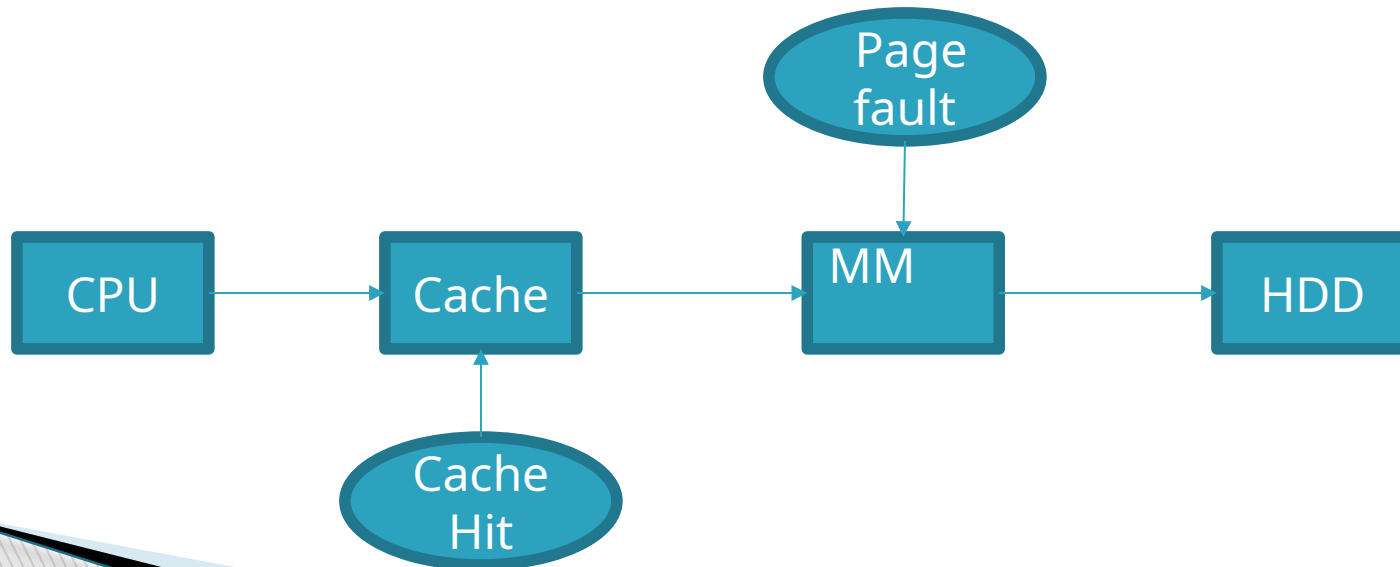
$$\begin{aligned}\text{Average memory access time} &= 10 * 0.9 + (1 - 0.9) * (50 + 10) \\ &= 9 + (0.1) * 60 \\ &= 9 + 6 = 15\text{ns (Ans.)}\end{aligned}$$

Memory Organization

- ▶ **Effective memory access time:**

- ▶ **Example 2:**

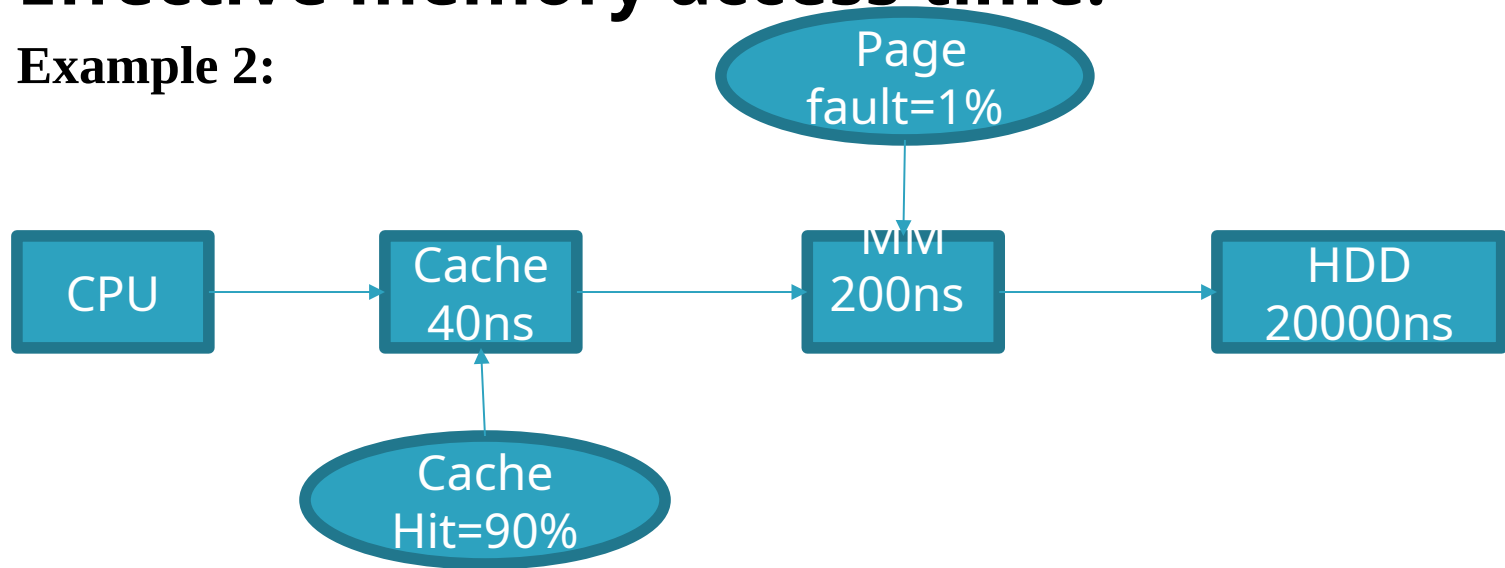
A computer with a single cache (access time 40ns) and main memory (access time 200ns) also uses the hard disk (average access time 0.02 ms) for virtual memory pages. If it is found that the cache hit rate is 90% and the page fault rate is 1%



Memory Organization

- ▶ **Effective memory access time:**

- ▶ **Example 2:**



- ▶
$$\begin{aligned} \text{EAT} &= 0.9 \times 40 + 0.1 \times 0.99(200 + 40) + 0.1 \times 0.01 \times (40 + 200 + 20000) \\ &= 36 + 23.76 + 20.24 \text{ ns} \\ &= 80 \text{ ns} \end{aligned}$$